

Here's an example of a software development anti-pattern.

Just like a Design Pattern, an anti-pattern has a name so we can create a shared vocabulary.

The problem and context, just like a Design Pattern description.

Tells you why the solution is attractive.

The bad, yet attractive, solution.

How to get to a good solution.

Example of where this anti-pattern has been observed.

Adapted from the Portland Pattern Repository's wiki at <https://wiki.c2.com/?WelcomeVisitors> where you'll find many anti-patterns and discussions.



Anti-Pattern

Name: Golden Hammer

Problem: You need to choose technologies for your development and you believe that exactly one technology must dominate the architecture.

Context: You need to develop some new system or piece of software that doesn't fit well with the technology that the development team is familiar with.

Forces:

- The development team is committed to the technology they know.
- The development team is not familiar with other technologies.
- Unfamiliar technologies are seen as risky.
- It is easy to plan and estimate for development using the familiar technology.

Supposed solution: Use the familiar technology anyway. The technology is applied obsessively to many problems, including places where it is clearly inappropriate.

Refactored solution: Expand the knowledge of developers through education, training, and book study groups that expose developers to new solutions.

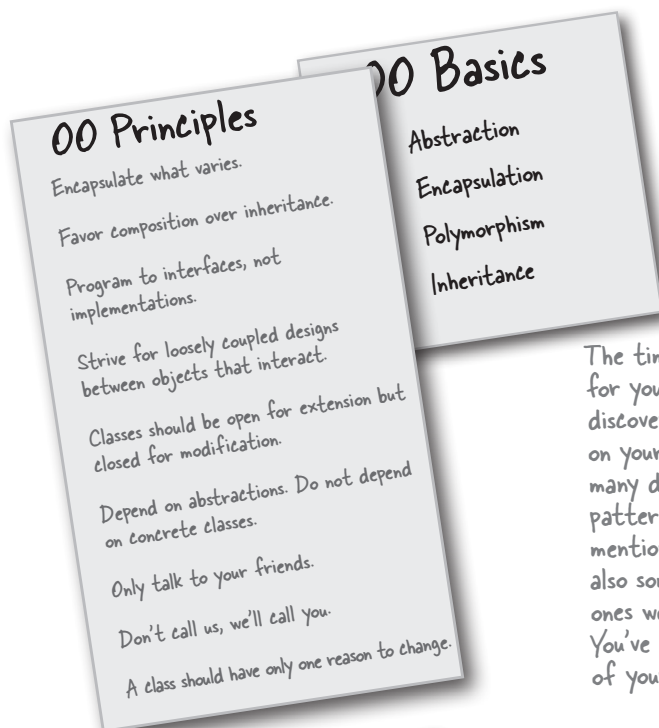
Examples:

Web companies keep using and maintaining their internal homegrown caching systems when open source alternatives are in use.



Tools for your Design Toolbox

You've reached that point where you've outgrown us. Now's the time to go out in the world and explore patterns on your own...



OO Basics

Abstraction
Encapsulation
Polymorphism
Inheritance

The time has come for you to go out and discover more patterns on your own. There are many domain-specific patterns we haven't even mentioned and there are also some foundational ones we didn't cover. You've also got patterns of your own to create.



Check out the Appendix; we'll give you a heads up on some more foundational patterns you'll probably want to have a look at.

BULLET POINTS

- Let Design Patterns emerge in your designs; don't force them in just for the sake of using a pattern.
- Design Patterns aren't set in stone; adapt and tweak them to meet your needs.
- Always use the simplest solution that meets your needs, even if it doesn't include a pattern.
- Study Design Patterns catalogs to familiarize yourself with patterns and the relationships among them.
- Pattern classifications (or categories) provide groupings for patterns. When they help, use them.
- You need to be committed to be a patterns writer: it takes time and patience, and you have to be willing to do lots of refinement.
- Remember, most patterns you encounter will be adaptations of existing patterns, not new patterns.
- Build your team's shared vocabulary. This is one of the most powerful benefits of using patterns.
- Like any community, the patterns community has its own lingo. Don't let that hold you back. Having read this book, you now know most of it.

Leaving Objectville...



Boy, it's been great having you in Objectville.

We're going to miss you, for sure. But don't worry—before you know it, the next Head First book will be out and you can visit again. What's the next book, you ask? Hmmm, good question! Why don't you help us decide? Send email to booksuggestions@wickedlysmart.com.

WHO DOES WHAT? SOLUTION

Match each pattern with its description:

Pattern	Description
Decorator	Wraps an object and provides a different interface to it.
State	Subclasses decide how to implement steps in an algorithm.
Iterator	Subclasses decide which concrete classes to create.
Facade	Ensures one and only one object is created.
Strategy	Encapsulates interchangeable behaviors and uses delegation to decide which one to use.
Proxy	Clients treat collections of objects and individual objects uniformly.
Factory Method	Encapsulates state-based behaviors and uses delegation to switch between behaviors.
Adapter	Provides a way to traverse a collection of objects without exposing its implementation.
Observer	Simplifies the interface of a set of classes.
Template Method	Wraps an object to provide new behavior.
Composite	Allows a client to create families of objects without specifying their concrete classes.
Singleton	Allows objects to be notified when state changes.
Abstract Factory	Wraps an object to control access to it.
Command	Encapsulates a request as an object.

Leftover Patterns



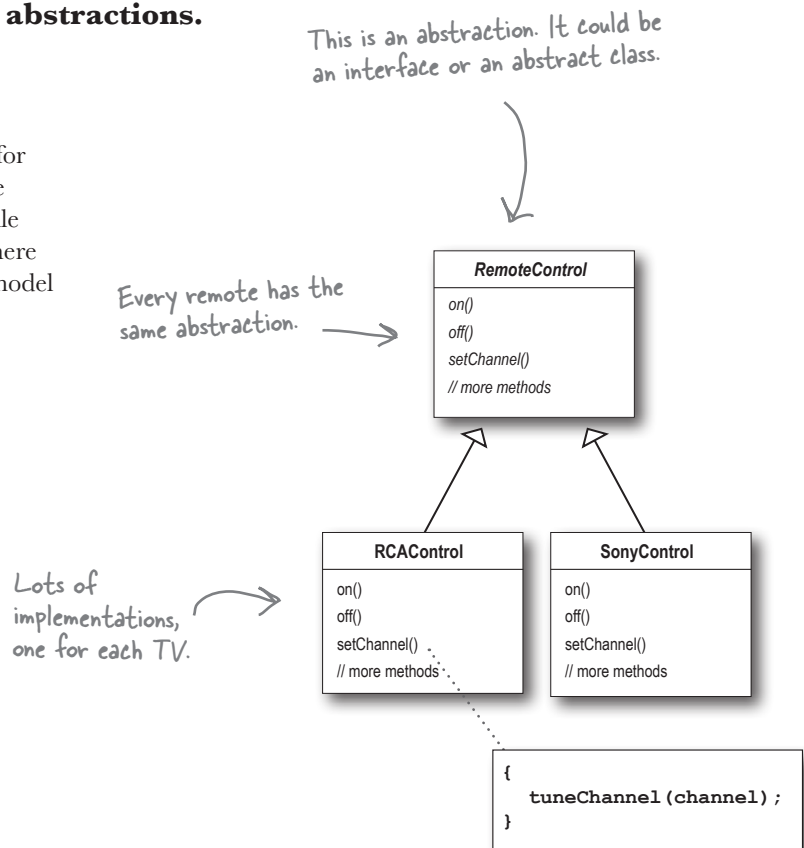
Not everyone can be the most popular. A lot has changed in the last 25+ years. Since *Design Patterns: Elements of Reusable Object-Oriented Software* first came out, developers have applied these patterns thousands of times. The patterns we summarize in this appendix are full-fledged, card-carrying, official GoF patterns, but aren't used as often as the patterns we've explored so far. But these patterns are awesome in their own right, and if your situation calls for them, you should apply them with your head held high. Our goal in this appendix is to give you a high-level idea of what these patterns are all about.

Bridge

Use the Bridge Pattern to vary not only your implementations, but also your abstractions.

A scenario

Imagine you're writing the code for a new ergonomic and user-friendly remote control for TVs. You already know that you've got to use good object-oriented techniques because while the remote is based on the same *abstraction*, there will be lots of *implementations*—one for each model of TV.



Your dilemma

You know that the remote's user interface won't be right the first time. In fact, you expect that the product will be refined many times as usability data is collected on the remote control.

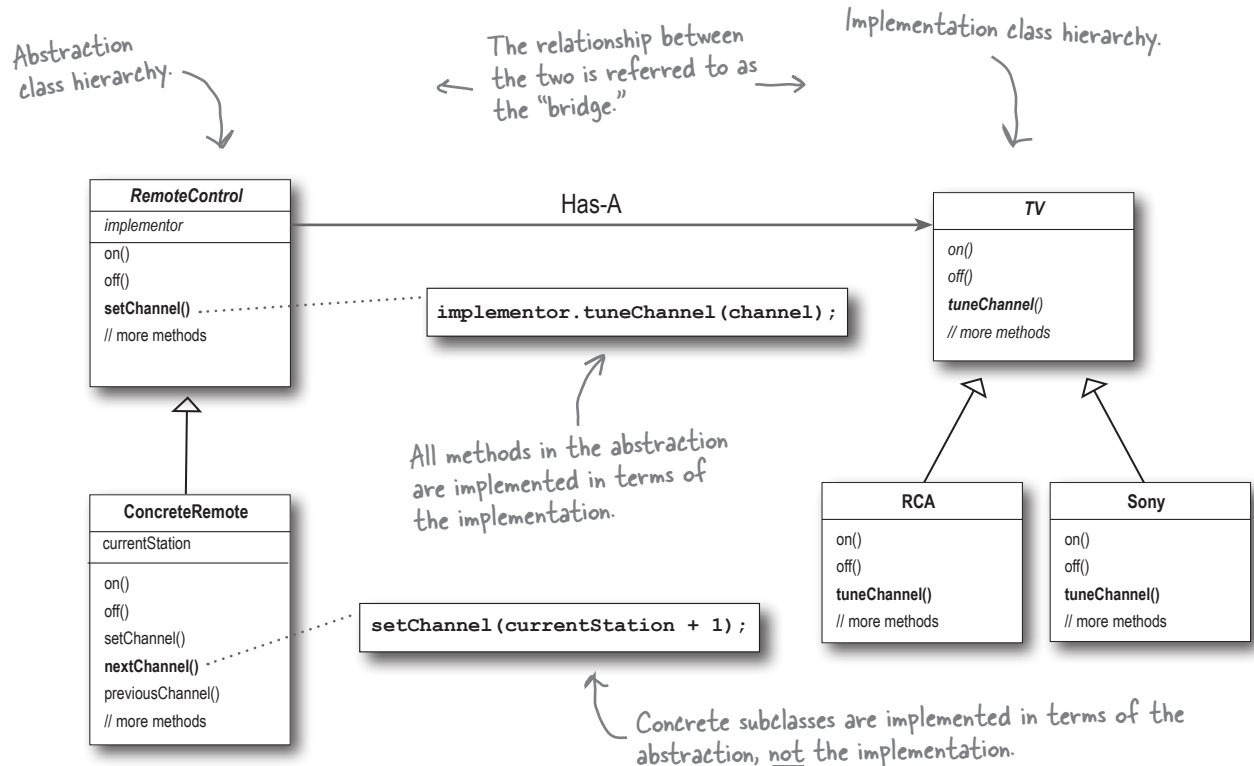
So your dilemma is that the remotes are going to change and the TVs are going to change. You've already *abstracted* the user interface so that you can vary the *implementation* over the many TVs your customers will own. But you are also going to need to *vary the abstraction* because it is going to change over time as the remote is improved based on the user feedback.

So how are you going to create an object-oriented design that allows you to vary the implementation *and* the abstraction?

Using this design we can vary only the TV implementation, not the user interface.

Why use the Bridge Pattern?

The Bridge Pattern allows you to vary the implementation *and* the abstraction by placing the two in separate class hierarchies.



Now you have two hierarchies, one for the remotes and a separate one for platform-specific TV implementations. The bridge allows you to vary either side of the two hierarchies independently.

Bridge Benefits

- Decouples an implementation so that it is not bound permanently to an interface.
- Abstraction and implementation can be extended independently.
- Changes to the concrete abstraction classes don't affect the client.

Bridge Uses and Drawbacks

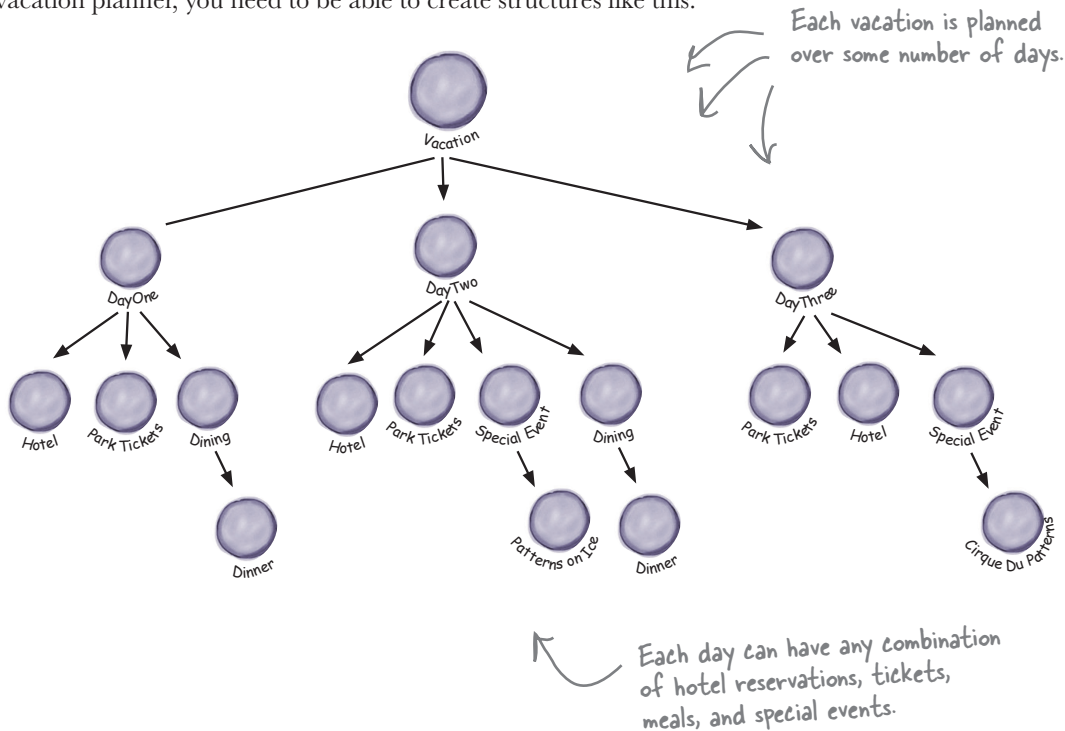
- Useful in graphics and windowing systems that need to run over multiple platforms.
- Useful any time you need to vary an interface and an implementation in different ways.
- Increases complexity.

Builder

Use the Builder Pattern to encapsulate the construction of a product and allow it to be constructed in steps.

A scenario

You've just been asked to build a vacation planner for Patternsland, a new theme park just outside of Objectville. Park guests can choose a hotel and various types of admission tickets, make restaurant reservations, and even book special events. To create a vacation planner, you need to be able to create structures like this:



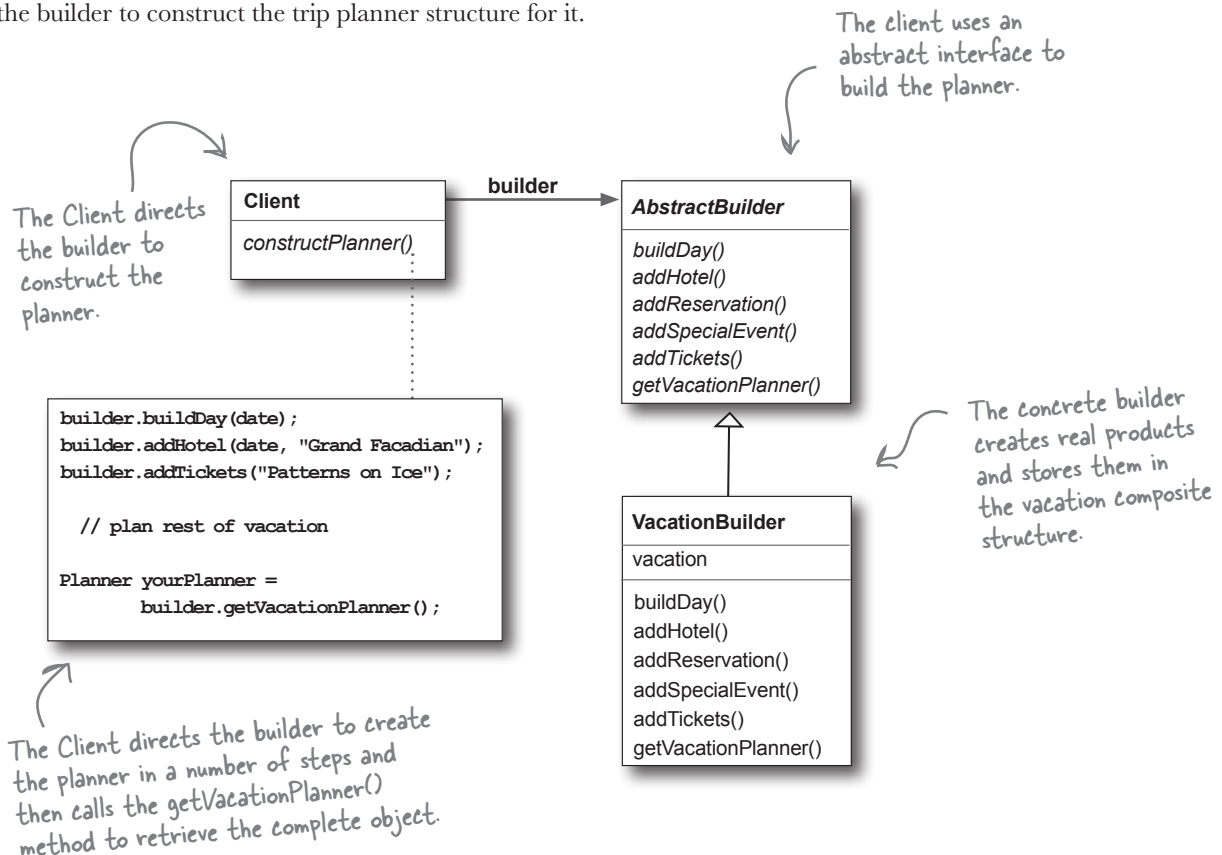
You need a flexible design

Each guest's planner can vary in the number of days and types of activities it includes. For instance, a local resident might not need a hotel, but wants to make dinner and special event reservations. Another guest might be flying into Objectville and needs a hotel, dinner reservations, and admission tickets.

So, you need a flexible data structure that can represent guest planners and all their variations; you also need to follow a sequence of potentially complex steps to create the planner. How can you provide a way to create the complex structure without mixing it with the steps for creating it?

Why use the Builder Pattern?

Remember Iterator? We encapsulated the iteration into a separate object and hid the internal representation of the collection from the client. It's the same idea here: we encapsulate the creation of the trip planner in an object (let's call it a builder), and have our client ask the builder to construct the trip planner structure for it.



Builder Benefits

- Encapsulates the way a complex object is constructed.
- Allows objects to be constructed in a multistep and varying process (as opposed to one-step factories).
- Hides the internal representation of the product from the client.
- Product implementations can be swapped in and out because the client only sees an abstract interface.

Builder Uses and Drawbacks

- Often used for building composite structures.
- Constructing objects requires more domain knowledge of the client than when using a Factory.

Chain of Responsibility

Use the Chain of Responsibility Pattern when you want to give more than one object a chance to handle a request.

A scenario

Mighty Gumball has been getting more email than they can handle since the release of the Java-powered Gumball Machine. From their own analysis they get four kinds of email: fan mail from customers that love the new 1-in-10 game, complaints from parents whose kids are addicted to the game, requests to put machines in new locations, and a fair amount of spam.

All fan mail should go straight to the CEO, all complaints should go to the legal department, and all requests for new machines should go to business development. Spam should be deleted.

Your task

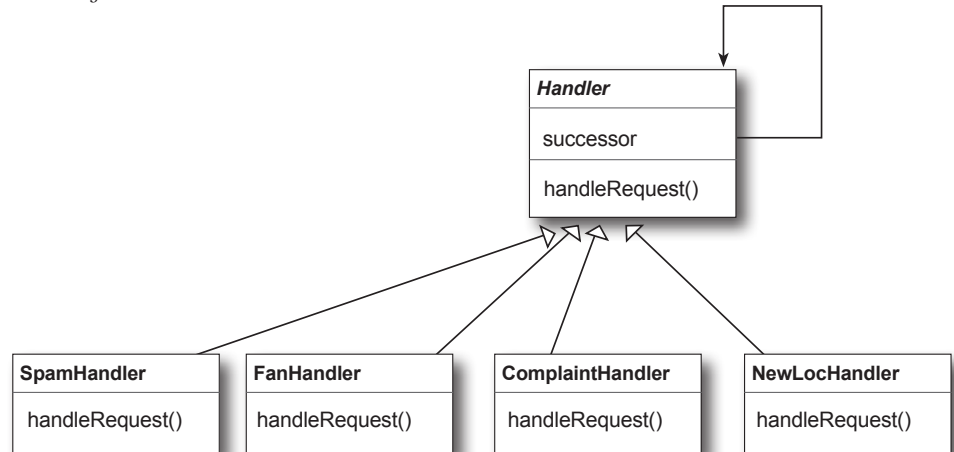
Mighty Gumball has already written some AI detectors that can tell if an email is spam, fan mail, a complaint, or a request, but they need you to create a design that can use the detectors to handle incoming email.



How to use the Chain of Responsibility Pattern

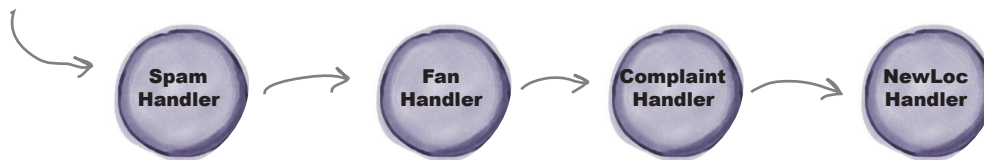
With the Chain of Responsibility Pattern, you create a chain of objects to examine requests. Each object in turn examines a request and either handles it or passes it on to the next object in the chain.

Each object in the chain acts as a handler and has a successor object. If it can handle the request, it does; otherwise, it forwards the request to its successor.



As email is received, it is passed to the first handler: SpamHandler. If the SpamHandler can't handle the request, it is passed on to the FanHandler. And so on...

Each email is passed to the first handler.



Email is not handled if it falls off the end of the chain — although you can always implement a catch-all handler.

Chain of Responsibility Benefits

- Decouples the sender of the request and its receivers.
- Simplifies your object because it doesn't have to know the chain's structure and keep direct references to its members.
- Allows you to add or remove responsibilities dynamically by changing the members or order of the chain.

Chain of Responsibility Uses and Drawbacks

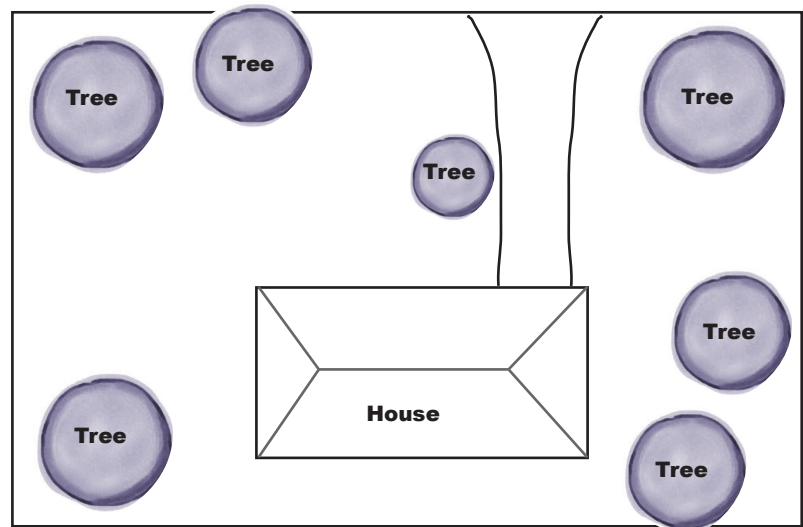
- Commonly used in Windows systems to handle events like mouse clicks and keyboard events.
- Execution of the request isn't guaranteed; it may fall off the end of the chain if no object handles it (this can be an advantage or a disadvantage).
- Can be hard to observe and debug at runtime.

Flyweight

Use the Flyweight Pattern when one instance of a class can be used to provide many virtual instances.

A scenario

You want to add trees as objects in your new landscape design application. In your application, trees don't really do very much; they have an X-Y location, and they can draw themselves dynamically, depending on how old they are. The thing is, a user might want to have lots and lots of trees in one of their home landscape designs. It might look something like this:



Each Tree instance maintains its own state.

```
Tree
xCoord
yCoord
age

display() {
    // use X-Y coords
    // & complex age
    // related calcs
}
```

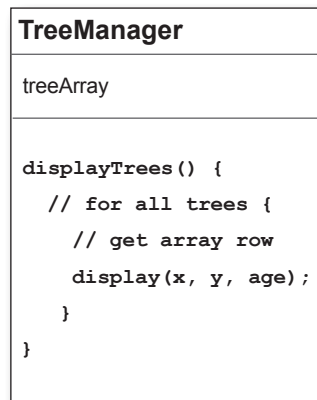
Your big client's dilemma

You have a key client you've been pitching for months. They're going to buy 1,000 seats of your application, and they're using your software to do the landscape design for huge planned communities. After using your software for a week, your client is complaining that when they create large groves of trees, the app starts getting sluggish...

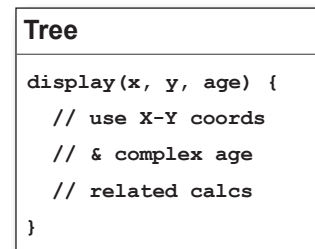
Why use the Flyweight Pattern?

What if, instead of having thousands of Tree objects, you could redesign your system so that you've got only one instance of Tree, and a client object that maintains the state of ALL your trees? That's the Flyweight!

All the state, for ALL of your virtual Tree objects, is stored in this 2D array.



One, single, state-free Tree object.



Flyweight Benefits

- Reduces the number of object instances at runtime, saving memory.
- Centralizes state for many “virtual” objects into a single location.

Flyweight Uses and Drawbacks

- The Flyweight is used when a class has many instances, and they can all be controlled identically.
- A drawback of the Flyweight Pattern is that once you've implemented it, single, logical instances of the class will not be able to behave independently from the other instances.

Interpreter

Use the Interpreter Pattern to build an interpreter for a language.

A scenario

Remember the Duck Simulator? You have a hunch it would also make a great educational tool for children to learn programming. Using the simulator, each child gets to control one duck with a simple language. Here's an example of the language:

```
right;
while (daylight) fly;
quack;
```

Turn the duck right.

Fly all day...

...and then quack.

Now, remembering how to create grammars from one of your old introductory programming classes, you write out the grammar:

```
expression ::= <command> | <sequence> | <repetition>
sequence  ::= <expression> ';' <expression>
command  ::= right | quack | fly
repetition ::= while '(' <variable> ')' <expression>
variable  ::= [A-Z,a-z] +
```

A program is an expression consisting of sequences of commands and repetitions ("while" statements).

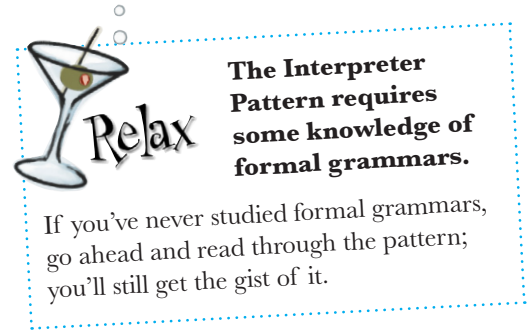
A sequence is a set of expressions separated by semicolons.

We have three commands: right, quack, and fly.

A while statement is just a conditional variable and an expression.

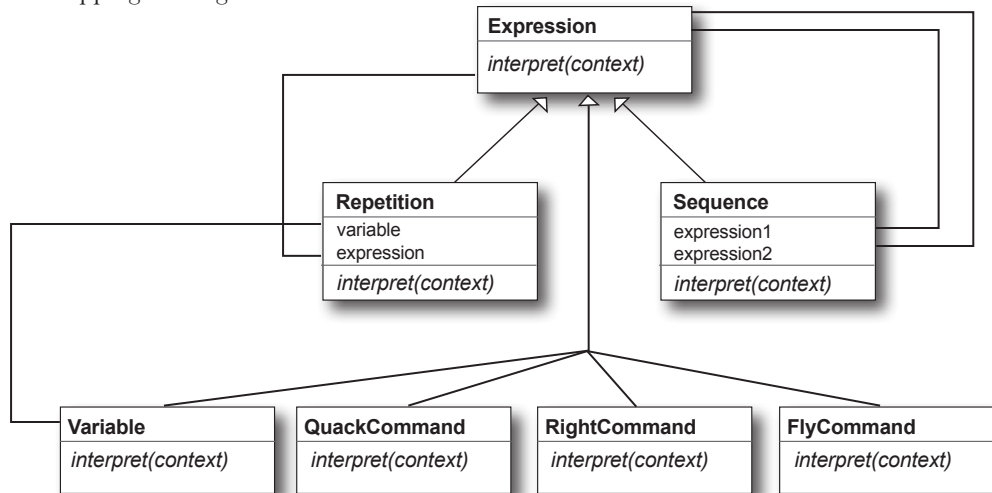
Now what?

You've got a grammar; now all you need is a way to represent and interpret sentences in the grammar so that the students can see the effects of their programming on the simulated ducks.



How to implement an interpreter

When you need to implement a simple language, the Interpreter Pattern defines a class-based representation for its grammar along with an interpreter to interpret its sentences. To represent the language, you use a class to represent each rule in the language. Here's the duck language translated into classes. Notice the direct mapping to the grammar.



To interpret the language, call the `interpret()` method on each expression type. This method is passed a context—which contains the input stream of the program we're parsing—and matches the input and evaluates it.

Interpreter Benefits

- Representing each grammar rule in a class makes the language easy to implement.
- Because the grammar is represented by classes, you can easily change or extend the language.
- By adding methods to the class structure, you can add new behaviors beyond interpretation, like pretty printing and more sophisticated program validation.

Interpreter Uses and Drawbacks

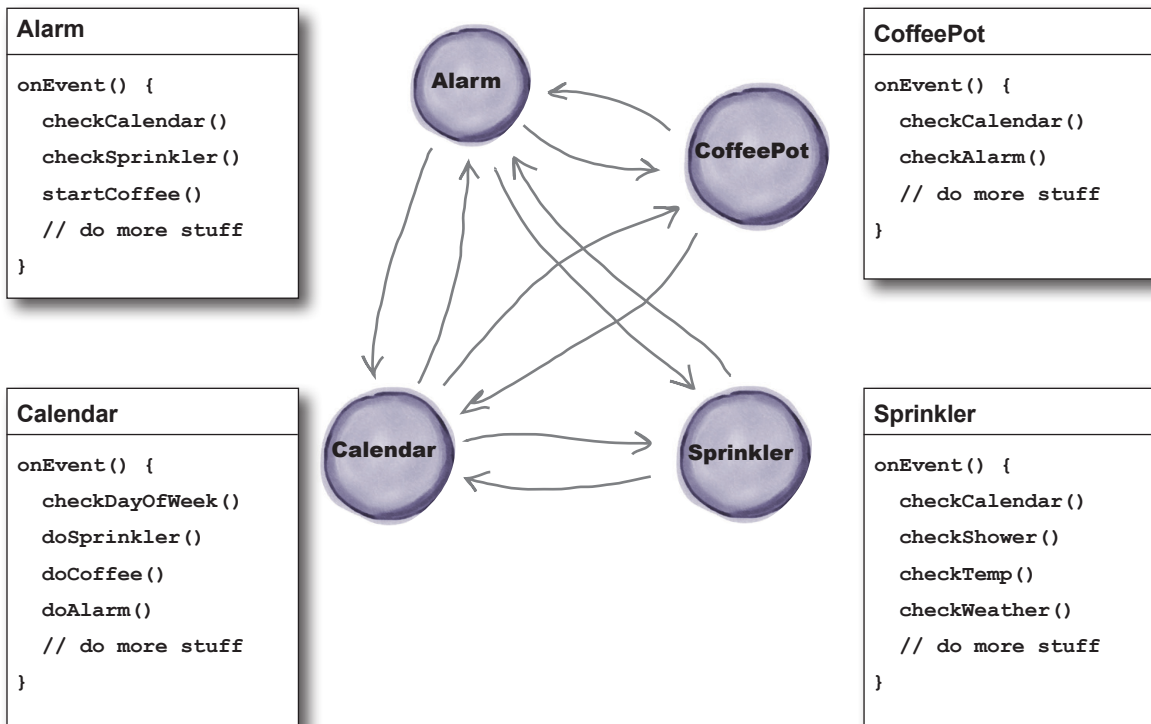
- Use Interpreter when you need to implement a simple language.
- Appropriate when you have a simple grammar and simplicity is more important than efficiency.
- Used for scripting and programming languages.
- This pattern can become cumbersome when the number of grammar rules is large. In these cases a parser/compiler generator may be more appropriate.

Mediator

Use the Mediator Pattern to centralize complex communications and control between related objects.

A scenario

Bob has an automated home, thanks to the good folks at HouseOfTheFuture. All of his appliances are designed to make his life easier. When Bob stops hitting the snooze button, his alarm clock tells the coffee maker to start brewing. Even though life is good for Bob, he and other customers are always asking for lots of new features: No coffee on the weekends... Turn off the sprinkler 15 minutes before a shower is scheduled... Set the alarm early on trash days...



HouseOfTheFuture's dilemma

It's getting really hard to keep track of which rules reside in which objects, and how the various objects should relate to each other.

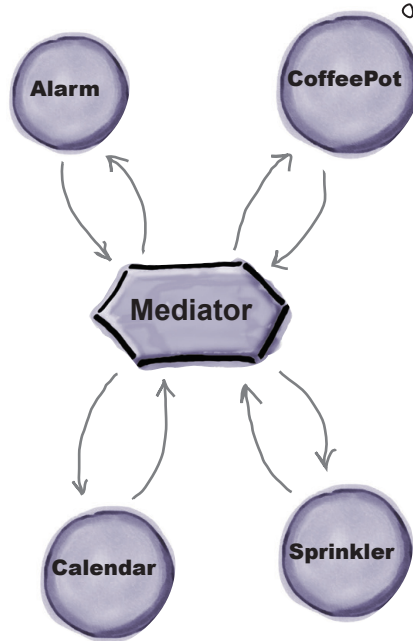
Mediator in action...

With a Mediator added to the system, all of the appliance objects can be greatly simplified:

- They tell the Mediator when their state changes.
- They respond to requests from the Mediator.

Before we added the Mediator, all of the appliance objects needed to know about each other; that is, they were all tightly coupled. With the Mediator in place, the appliance objects are all completely decoupled from each other.

The Mediator contains all of the control logic for the entire system. When an existing appliance needs a new rule, or a new appliance is added to the system, you'll know that all of the necessary logic will be added to the Mediator.



It's such a relief, not having to figure out that Alarm clock's picky rules!

Mediator

```

if (alarmEvent) {
    checkCalendar()
    checkShower()
    checkTemp()
}
if (weekend) {
    checkWeather()
    // do more stuff
}
if (trashDay) {
    resetAlarm()
    // do more stuff
}
  
```

Mediator Benefits

- Increases the reusability of the objects supported by the Mediator by decoupling them from the system.
- Simplifies maintenance of the system by centralizing control logic.
- Simplifies and reduces the variety of messages sent between objects in the system.

Mediator Uses and Drawbacks

- The Mediator is commonly used to coordinate related GUI components.
- A drawback of the Mediator Pattern is that without proper design, the Mediator object itself can become overly complex.

Memento

Use the Memento Pattern when you need to be able to return an object to one of its previous states; for instance, if your user requests an “undo.”

A scenario

Your interactive role-playing game is hugely successful, and has created a legion of addicts, all trying to get to the fabled “level 13.” As users progress to more challenging game levels, the odds of encountering a game-ending situation increase. Fans who have spent days progressing to an advanced level are understandably miffed when their character gets snuffed, and they have to start all over. The cry goes out for a “save progress” command, so that players can store their game progress and at least recover most of their efforts when their character is unfairly extinguished. The “save progress” function needs to be designed to return a resurrected player to the last level she completed successfully.



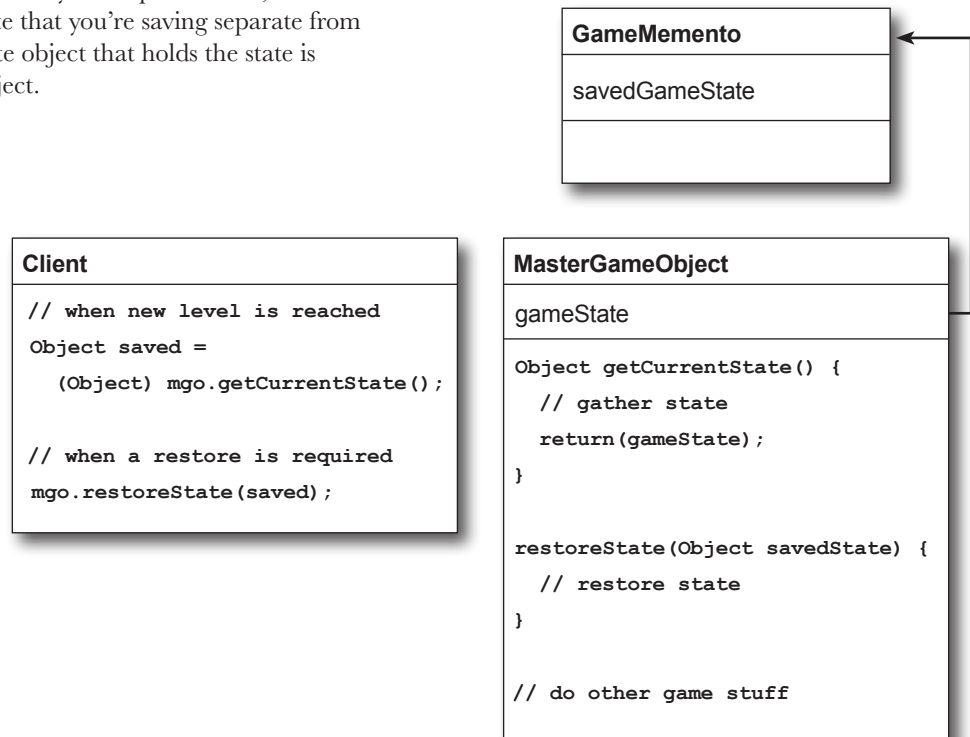
The Memento at work

The Memento has two goals:

- Saving the important state of a system's key object
- Maintaining the key object's encapsulation

Keeping the Single Responsibility Principle in mind, it's also a good idea to keep the state that you're saving separate from the key object. This separate object that holds the state is known as the Memento object.

While this isn't a terribly fancy implementation, notice that the Client has no access to the Memento's data.



Memento Benefits

- Keeping the saved state external from the key object helps to maintain cohesion.
- Keeps the key object's data encapsulated.
- Provides easy-to-implement recovery capability.

Memento Uses and Drawbacks

- The Memento is used to save state.
- A drawback to using Memento is that saving and restoring state can be time-consuming.
- In Java systems, consider using Serialization to save a system's state.

Prototype

Use the Prototype Pattern when creating an instance of a given class is either expensive or complicated.

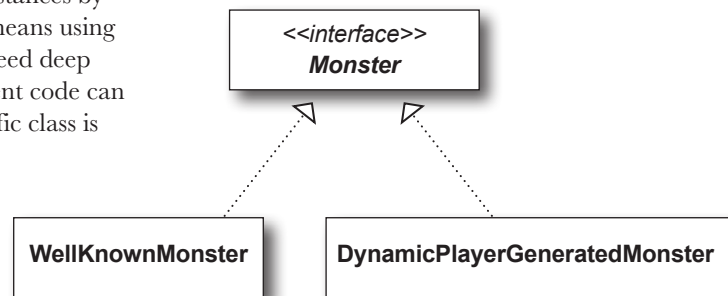
A scenario

Your interactive role-playing game has an insatiable appetite for monsters. As your heroes make their journey through a dynamically created landscape, they encounter an endless chain of foes that must be subdued. You'd like the monster's characteristics to evolve with the changing landscape. It doesn't make a lot of sense for bird-like monsters to follow your characters into underseas realms. Finally, you'd like to allow advanced players to create their own custom monsters.



Prototype to the rescue

The Prototype Pattern allows you to make new instances by copying existing instances. (In Java this typically means using the `clone()` method, or deserialization when you need deep copies.) A key aspect of this pattern is that the client code can make new instances without knowing which specific class is being instantiated.



```

MonsterMaker

makeRandomMonster() {
    Monster m =
        MonsterRegistry.getMonster();
}
  
```

The client needs a new monster appropriate to the current situation. (The client won't know what kind of monster he gets.)

```

MonsterRegistry

Monster getMonster() {
    // find the correct monster
    return correctMonster.clone();
}
  
```

The registry finds the appropriate monster, makes a clone of it, and returns the clone.

Prototype Benefits

- Hides the complexities of making new instances from the client.
- Provides the option for the client to generate objects whose type is not known.
- In some circumstances, copying an object can be more efficient than creating a new object.

Prototype Uses and Drawbacks

- Prototype should be considered when a system must create new objects of many types in a complex class hierarchy.
- A drawback to using Prototype is that making a copy of an object can sometimes be complicated.

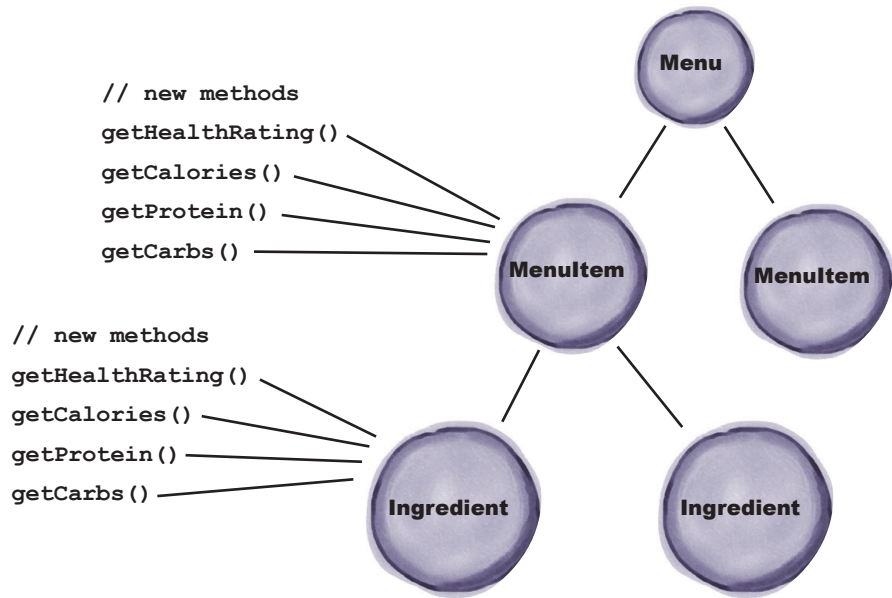
Visitor

Use the Visitor Pattern when you want to add capabilities to a composite of objects and encapsulation is not important.

A scenario

Customers who frequent the Objectville Diner and Objectville Pancake House have recently become more health conscious. They are asking for nutritional information before ordering their meals. Because both establishments are so willing to create special orders, some customers are even asking for nutritional information on a per-ingredient basis.

Lou's proposed solution:

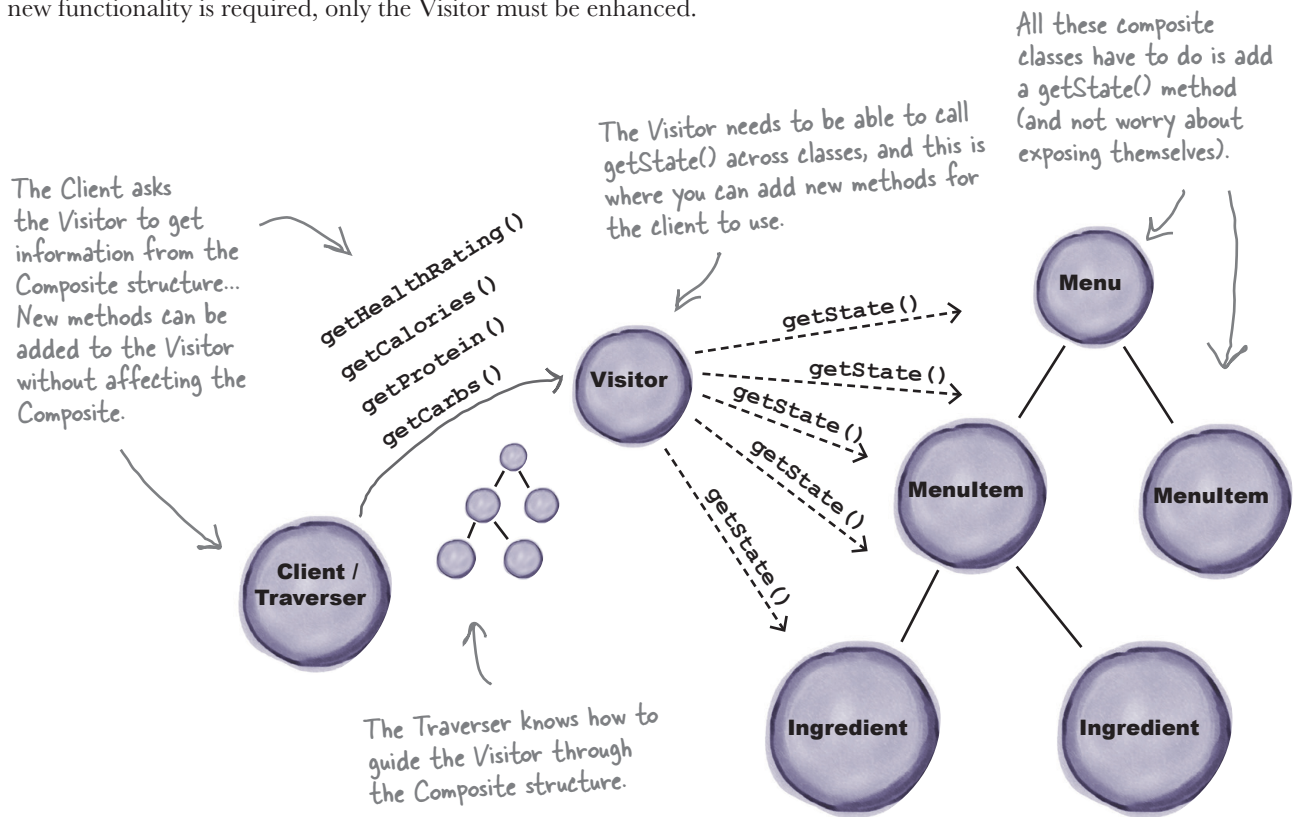


Mel's concerns...

“Boy, it seems like we’re opening Pandora’s box. Who knows what new method we’re going to have to add next, and every time we add a new method we have to do it in two places. Plus, what if we want to enhance the base application with, say, a recipes class? Then we’ll have to make these changes in three different places...”

The Visitor drops by

The Visitor works hand in hand with a Traverser. The Traverser knows how to navigate to all of the objects in a Composite. The Traverser guides the Visitor through the Composite so that the Visitor can collect state as it goes. Once state has been gathered, the Client can have the Visitor perform various operations on the state. When new functionality is required, only the Visitor must be enhanced.



Visitor Benefits

- Allows you to add operations to a Composite structure without changing the structure itself.
- Adding new operations is relatively easy.
- The code for operations performed by the Visitor is centralized.

Visitor Drawbacks

- The Composite classes' encapsulation is broken when the Visitor is used.
- Because the traversal function is involved, changes to the Composite structure are more difficult.

Index

A

- AbstractButton class 65
- abstract class 128, 292, 293
- Abstract Factory Pattern
 - about 153
 - building ingredient factories 146–148, 167
 - combining patterns 502–505, 548
 - defined 156–157
 - exercise matching description of 574, 596
 - Factory Method Pattern and 158–161
 - implementing 158
 - interview with 158–159
- AbstractList 309
- abstract superclasses 12
- Adapter Pattern
 - about 243–244
 - adapting to Iterator Enumeration interface 251
 - combining patterns 498–499
 - dealing with remove() method 252
 - Decorator Pattern vs. 254–255
 - defined 245
 - designing Adapter 251
 - exercises for 256, 275, 375, 379, 481, 574, 596
 - Facade Pattern vs. 262
 - in Model-View-Controller 540
 - object and class adapters 246–249
 - Proxy Pattern vs. 466
 - simple real world adapters 250
 - writing Enumeration Iterator Adapter 252–253
- adapters, OO (Object-Oriented)
 - about 238–239
 - creating Two Way Adapters 244
 - in action 240–241
 - object, class object and class 246–249
 - test driving 242
- aggregates 327, 338
- album covers, displaying using Proxy Pattern
 - about 458
 - code for 489–492
 - designing Virtual Proxy 459
 - reviewing process 465
 - testing viewer 464
 - writing Image Proxy 460–463
- Alexander, Christopher
 - A Pattern Language 588
 - The Timeless Way of Building 588
- algorithms, encapsulating
 - about 277
 - abstracting prepareRecipe() 284–287
 - Strategy Pattern and 24
 - Template Method Pattern and
 - about 288–290
 - applets in 309
 - code up close 292–293
 - defined 291
 - The Hollywood Principle and 298–300
 - hooks in 293–295
 - in real world 301
 - sorting with 302–307
 - Swing and 308
 - testing code 296
- algorithms, family of 22
- Anti-Patterns 592–593
- Applicability section, in pattern catalog 571
- Application Patterns 590
- Architectural Patterns 590
- ArrayList, arrays and 320–325, 351
- arrays
 - iteration and 325–326, 345
 - iterator and hasNext() method with 328
 - iterator and next() method with 328

B

behavioral patterns categories, Design Patterns 576, 578–579

behaviors

- classes as 14
- classes extended to incorporate new 86
- declaring variables 15
- delegating to decorated objects while adding 90
- designing 11–12
- encapsulating 11, 22
- implementing 11, 13
- separating 10
- setting dynamically 20–21

Be the JVM solution exercises, dealing with multithreading 179–180, 188

Bridge Pattern 598–599

Builder Pattern 600–601

Business Process Patterns 591

C

Caching Proxy, as form of Virtual Proxy 466, 482

Café Menu, integrating into framework (Iterator Pattern) 347

Chain of Responsibility Pattern 602–603

change

- as the one constant in software development 8
- identifying 54
- iteration and 340

chocolate factory example, using Singleton Pattern 175–176, 183

class adapters, object vs. 246–249

class design, of Observer Pattern 51–52

classes. *See also* subclasses

- abstract 128, 292, 293
- adapter 244, 274
- Adapter Pattern 245
- altering decorator 108
- as behaviors 14
- collection 352
- creating 10
- extended to incorporate new behaviors 86

Factory Method Pattern creator and product 131–132

having single responsibility 340–341

high-level component 139

identifying as Proxy class 480

relationships between 22

state

defining 395

implementing 397, 400–405, 409

increasing number in design of 408

reworking 398–399

state transitions in 408

using composition with 23

using instance variables instead of 82–83

using instead of Singletons static 184

using new operator for instantiating concrete 110–113

Classification section, in pattern catalog 571

classloaders, using with Singletons 184

class patterns, Design Patterns 577

client heap 433–436

client helper (stubs), in RMI 436–437, 440, 442–444, 453–454

Code Magnets exercise

for DinerMenu Iterator 354, 378

for Observer Pattern 70, 76

cohesion 340

Collaborations section, in pattern catalog 571

collection classes 352

collection of objects

abstracting with Iterator Pattern

about 317

adding Iterators 328–334

cleaning up code using `java.util.Iterator` 335–337

`remove()` method in 334

implementing Iterators for 327

integrating into framework 347

meaning of 327

using Composite Pattern

about 363

implementing components 364–366

testing code 368–370

tree structure 360–362, 368

using whole-part relationships 372

Collections, Iterators and 353

Combining Patterns

- Abstract Factory Pattern 502–505
- Adapter Pattern 498–499
- class diagram for 518–519
- Composite Pattern 507–509
- Decorator Pattern 500–501
- Iterator Pattern 507
- Observer Pattern 510–516

command objects

- encapsulating requests to do something 196
- mapping 201
- using 204

Command Pattern

- command objects
 - building 203
 - encapsulating requests to do something 196
 - mapping 201
 - using 204
- defined 206–207
- dumb and smart command objects 228
- exercise matching description of 574, 596
- home automation remote control
 - about 193
 - building 203–205, 235
 - class diagram 207
 - creating commands to be loaded 208–209
 - defining 206
 - designing 195–196
 - implementing 210–212
 - macro commands 225, 226–228, 236
 - mapping 201–202, 235
 - Null Object in 214
 - testing 204, 212–213, 227
 - undo commands 217–221, 223–224, 228, 236
 - vendor classes for 194
 - writing documentation 215
- logging requests using 230
- mapping 201–202, 235
- Null Object 214
- queuing requests using 229
- understanding 197–200

- compareTo() method 303

Complexity Hiding Proxy 483

- components of object 267–271

Composite Pattern

- combining patterns 507–509
- defined 360
- dessert submenu using
 - about 357
 - designing 363, 371
 - implementing 364–367
 - testing 368–370
- exercise matching description of 375, 379, 574, 596
- in Model-View-Controller 526–527, 543
- interview with 372–373
- on implementation issues 372–373
- safety vs. transparency 509
- transparency in 371
- tree structure of 360–362, 368

composition

- adding behavior at runtime 85
- favoring over inheritance 23, 85
- inheritance vs. 93
- object adapters and 249

compound patterns, using

- about 493–494
- Model-View-Controller
 - about 520–521, 523–525
 - Adapter Pattern 539
 - Beat model 529, 549–552
 - Composite Pattern 526–527, 543
 - controllers per view 543
 - Heart controller 541, 561
 - Heart model 539, 558–560
 - implementing controller 536–537, 556–557
 - implementing DJ View 528–535, 553–556
 - Mediator Pattern 543
 - model in 543
 - Observer Pattern 526–527, 531–533
 - song 520–521
 - state of model 543
 - Strategy Pattern 526–527, 536–537, 539, 558–560
 - testing 538
 - views accessing model state methods 543
- multiple patterns vs. 516

concrete classes

- deriving from 143
- Factory Pattern and 134
- getting rid of 116

- instantiating objects and 138
- using new operator for instantiating 110–113
- variables holding reference to 143

concrete creators 135

concrete implementation object, assigning, 12

concrete methods, as hooks 293–295

concrete subclasses 121–122, 297

Consequences section, in pattern catalog 571

controlling object access, using Proxy Pattern

- about 426–428
- Caching Proxy 466, 482
- Complexity Hiding Proxy 483
- Copy-On-Write Proxy 483
- Firewall Proxy 482
- Protection Proxy
 - about 469
 - creating dynamic proxy 474–478
 - implementing matchmaking service 471–472
 - protecting subjects 473
 - testing matchmaking service 479–480
 - using dynamic proxy 469–470
- Remote Proxy
 - about 429
 - adding to monitoring code 432
 - preparing for remote service 446–447
 - registering with RMI registry 448
 - reusing client for 449
 - reviewing process 453–455
 - role of 430–431
 - testing 450–452
 - wrapping objects and 468
- Smart Reference Proxy 482
- Synchronization Proxy 483
- Virtual Proxy
 - about 457
 - designing Virtual Proxy 459
 - reviewing process 465
 - testing 464
 - writing Image Proxy 460–463

Copy-On-Write Proxy 483

create method

- replacing new operator with 116
- static method vs. 115
- using subclasses with 121–122

creating static classes instead of Singleton 179–180

creational patterns category, Design Patterns 576, 578–579

creator classes, in Factory Method Pattern 131–132, 134–135

crossword puzzle 33, 74, 163, 187, 234, 273, 311, 374, 484

Cunningham, Ward 589

D

Decorator Pattern

- about 88–90, 104
- Adapter Pattern vs. 254–255
- combining patterns 500–501
- defined 91
- disadvantages of 101
- exercises for 256, 275, 481, 574, 596
- in Java I/O 100–101
- in Structural patterns category 577
- interview with 104
- Proxy Pattern vs. 466–468
- Starbuzz Coffee project
 - about 80–81
 - adding sizes to code 99
 - constructing drink orders 89–90
 - drawing beverage order process 94, 107
 - testing order code 98–99
 - writing code 95–97

decoupling, Iterator allowing 333, 337, 339, 351–352

delegation, adding behavior at runtime 85

dependence, in Observer Pattern 52

Dependency Inversion Principle (DIP) 139–143, 300

dependency rot 298

depend upon abstractions design principle 139

Design Patterns

- becoming writer of 573
- behavioral patterns category 576, 578–579
- categories of 576–579
- class patterns 577
- creational patterns category 576, 578–579
- defined 565–567
- discovering own 572
- exercise matching description of 596

- frameworks vs. 29
 - guide to better living with 564
 - implement on interface in 117
 - libraries vs. 29
 - object patterns 577
 - organizing 575
 - overusing 584
 - resources for 588–589
 - rule of three applied to 573
 - structural patterns category 576, 578–579
 - thinking in patterns 580–581
 - using 29, 582, 584
 - your mind on patterns 583
 - Design Patterns- Reusable Object-Oriented Software (Gamma et al.) 588
 - design principles
 - Dependency Inversion Principle 139–143
 - depend upon abstractions 139
 - Encapsulate what varies 9, 73, 75, 136, 393
 - Favor composition over inheritance 23, 73, 75, 393
 - The Hollywood Principle 298–300
 - One Class, One Responsibility Principle 184, 340, 371
 - one instance. *See* Singleton Pattern
 - Open-Closed Principle 355, 392
 - Principle of Least Knowledge 267–271
 - Program to an interface, not an implementation 11–12, 73, 75–76, 337
 - Single Responsibility Principle (SRP) 340–341
 - using Observer Pattern 73, 75
 - Design Puzzles
 - drawing class diagram making use of view and controller 536, 548
 - drawing parallel set of classes 133, 165
 - drawing state diagram 391, 420
 - of classes and interfaces 25, 34
 - redesigning classes to remove redundancy 281, 278–283
 - redesigning Image Proxy 463, 486
 - dessert submenu, using Composite Pattern
 - about 357
 - designing 363, 371
 - implementing 364–367
 - testing 368–370
 - diner menus, merging (Iterator Pattern)
 - about 318–319
 - adding Iterators 328–334
 - cleaning up code using `java.util.Iterator` 335–337
 - encapsulating Iterator 325–326
 - implementing Iterators for 327
 - implementing of 320–325
 - DIP (Dependency Inversion Principle) 139–143, 300
 - DJ View 528–535, 549–561
 - Domain-Specific Patterns 590
 - double-checked locking, reducing use of synchronization using 182
 - Duck Magnets exercises, object and class object and class adapters 247–248
 - duck simulator, rebuilding
 - about 495–497
 - adding Abstract Factory Pattern 502–505, 548
 - adding Adapter Pattern 498–499
 - adding Composite Pattern 507–509
 - adding Decorator Pattern 500–501
 - adding Iterator Pattern 507
 - adding Observer Pattern 510–516
 - class diagram 518–519
 - dumb command objects 228
 - dynamic aspect of dynamic proxies 480
 - dynamic proxy 469–470, 474–478
- ## E
- Encapsulate what varies design principle 9, 73, 75, 136, 393
 - encapsulating
 - behavior 11
 - code 22–23, 114–115, 136
 - iteration 325–326
 - method invocation 191, 206
 - object construction 600
 - requests 206
 - encapsulating algorithms
 - about 277
 - abstracting `prepareRecipe()` 284–287
 - Template Method Pattern and
 - about 288–290
 - `AbstractList` and 309
 - code up close 292–293
 - defined 291
 - The Hollywood Principle and 298–300

- hooks in 293–295
 - in real world 301
 - sorting with 302–307
 - Swing and 308
 - testing code 296
 - encapsulating subsystem, Facades 262
 - Enumeration
 - about 250
 - adapting to Iterator 251
 - java.util.Enumeration as older implementation of Iterator 250, 342
 - remove() method and 252
 - writing Adapter that adapts Iterator to 253, 275
 - exercises
 - Be the JVM solution, dealing with multithreading 179–180, 188
 - Code Magnets
 - for DinerMenu Iterator 354, 378
 - for Observer Pattern 70, 76
 - dealing with multithreading 247–248
 - Design Puzzles
 - drawing class diagram making use of view and controller 536, 548
 - drawing parallel set of classes 133, 165
 - drawing state diagram 391, 420
 - of classes and interfaces 25, 34
 - redesigning classes to remove redundancy 281–282
 - redesigning Image Proxy 463, 486
 - Duck Magnets exercises, object and class object and class adapters 247
 - implementing Iterator 329
 - implementing undo button for macro command 228, 236
 - Sharpen Your Pencil
 - altering decorator classes 99, 108
 - annotating Gumball Machine states 405, 423
 - annotating state diagram 396, 422
 - building ingredient factory 148, 167
 - changing classes for Decorator Pattern 512, 546
 - changing code to fit framework in Iterator Pattern 347, 377
 - choosing descriptions of state of implementation 392, 421
 - class diagram for implementation of prepareRecipe() 286, 314
 - code not using factories 137, 166
 - creating commands for off buttons 226, 236
 - creating heat index 62
 - determining classes violating Principle of Least Knowledge 270, 274
 - drawing beverage order process 107
 - fixing Chocolate Boiler code 183, 190
 - identifying factors influencing design 84
 - implementing garage door command 205, 235
 - implementing state classes 402, 421
 - making pizza store 124, 164
 - matching patterns with categories 575–577
 - method for refilling gumball machine 417, 424
 - on adding behaviors 14
 - on implementation of printmenu() 324, 377
 - on inheritance 5, 35
 - sketching out classes 55
 - things driving change 8, 35
 - turning class into Singleton 176, 189
 - weather station SWAG 42, 75
 - writing Abstract Factory Pattern 505, 548
 - writing classes for adapters 244, 274
 - writing dynamic proxy 478, 487
 - writing Flock observer code 514, 547
 - writing methods for classes 83, 106
 - Who Does What
 - matching objects and methods to Command Pattern 202, 235
 - matching patterns with its intent 256, 275
 - matching pattern with description 300, 314, 375, 379, 418, 424, 481, 488, 574, 596
 - writing Adapter that adapts Iterator to Enumeration 253, 275
 - writing handler for matchmaking service 477, 486
 - external iterators 342
- ## F
- Facade Pattern
 - about 256
 - Adapter Pattern vs. 262
 - advantages 262